Documentazione del Progetto Java

Corso di Object Orientation

Autore: Gino Pandozzi-Trani
Matricola: N86003116

---

20 luglio 2025

Indice

1 Contesto e Introduzione **2**

2 Funzionalità Implementate **2**

3 Architettura del Sistema **2**

4 Struttura del Codice **3**

5 Diagrammi UML **4**

5.1 Class Diagram 4

5.2 Sequence Diagram 4

1 Contesto e Introduzione

Descrizione generale

Il progetto consiste nello sviluppo di un sistema informativo, composto da una base di dati relazionale e un applicativo Java dotato di GUI (Swing o JavaFX), per la gestione di un negozio di ortofrutta. Il sistema permette di gestire le vendite e la disponibilità dei prodotti, identificando le tipologie (frutta, verdura, farinacei, latticini, uova e confezionati), ognuna con caratteristiche specifiche (ad esempio, la frutta e la verdura hanno una data di raccolta, i latticini hanno sia una data di mungitura del latte che una data di produzione, ecc.).

2 Funzionalità Implementate

Funzionalità principali

- Gestione prodotti (inserimento, modifica, cancellazione, ricerca)
- Gestione clienti e punti fedeltà
- Visualizzazione vendite e dipendenti
- Classifica vendite per dipendente e per introito
- Interfaccia grafica completa con Swing

Ogni cliente è registrato e possiede una tessera punti che lo identifica presso il fruttivendolo: per ogni acquisto riceve il 10% del valore della spesa in punti fedeltà. Il sistema consente la ricerca dei clienti, differenziandoli in base alle categorie di prodotti acquistati e alla quantità di punti ottenuti per ciascuna categoria.

Traccia d'esame: Sistema di gestione per un negozio di alimentari

Obiettivo della traccia

Per gruppi da 3: il fruttivendolo ha un certo numero di dipendenti e ad ogni vendita è associato il dipendente che l'ha portata a termine. È quindi possibile ricercare quale dipendente ha effettuato il maggior numero di vendite in un determinato periodo, differenziandolo da quello che ha portato il maggior introito.

3 Architettura del Sistema

Il progetto adotta il pattern architetturale Model-View-Controller (MVC), separando chiaramente la logica dell'applicazione, l'interfaccia grafica e l'accesso ai dati.

- **Model**: gestisce le entità come Prodotto, Cliente, Dipendente, Vendita, ecc.
- **View**: realizza la GUI in Swing, con schermate per l'inserimento, la modifica e la visualizzazione dei dati.

- **Controller**: gestisce l'interazione tra vista e modello, e collega l'interfaccia grafica alle funzionalità.

4 Struttura del Codice

Il progetto è suddiviso nei seguenti package, in linea con la struttura reale:

- `model` – Classi delle entità principali (Prodotto, Cliente, Dipendente, Ordine, Tessera, ecc.) e relativa logica di dominio.
- `controller` – Logica applicativa, gestione degli eventi e coordinamento tra GUI e modello.
- `daoimplementation` e `daointerface` – Interfacce e implementazioni per l'accesso ai dati tramite JDBC.
- `dbconfiguration` – Gestione della connessione al database e configurazione.
- `gui` – Componenti Swing per l'interfaccia grafica (schermate di inserimento, modifica, visualizzazione, ecc.).

Esempio di codice (classe Cliente)

Esempio: classe Cliente e calcolo punti fedeltà

La classe `Cliente` rappresenta un cliente registrato, dotato di una tessera punti. Il metodo `calcolaPunti` restituisce i punti fedeltà maturati in base alla spesa effettuata (10% del valore).

```java
public class Cliente {
    private String nome;
    private String cognome;
    private String tesseraPunti;

    public Cliente(String nome, String cognome, String
       tesseraPunti) {
        this.nome = nome;
        this.cognome = cognome;
        this.tesseraPunti = tesseraPunti;
    }

    /**
     * Calcola i punti fedelta maturati in base alla spesa.
     * @param spesa importo della spesa effettuata
     * @return punti maturati (10 percento della spesa)
     */
    public int calcolaPunti(double spesa) {
        return (int)(spesa * 0.10);
    }

    // Getter e Setter
}
```

5 Diagrammi UML

5.1 Class Diagram

Il seguente diagramma delle classi rappresenta la struttura principale del sistema e le relazioni tra le entità fondamentali.

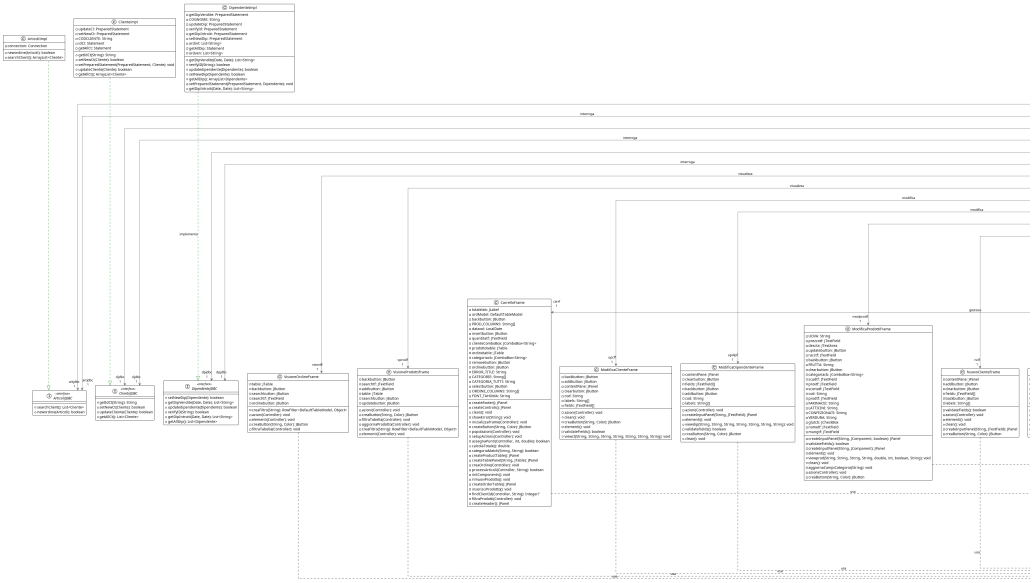

Figura 1: Diagramma delle classi (Class Diagram)

5.2 Sequence Diagram

Di seguito sono riportati i principali diagrammi di sequenza che illustrano il flusso delle operazioni più rilevanti del sistema.

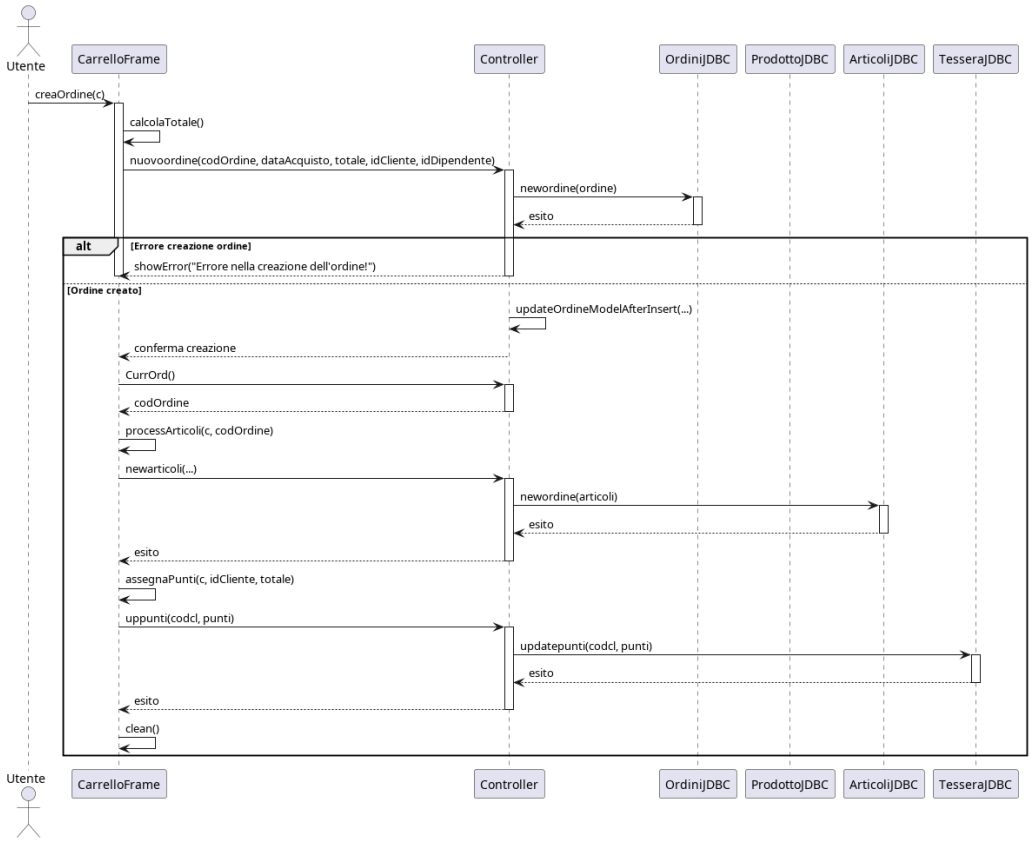

Figura 2: Diagramma di sequenza: Creazione Ordine

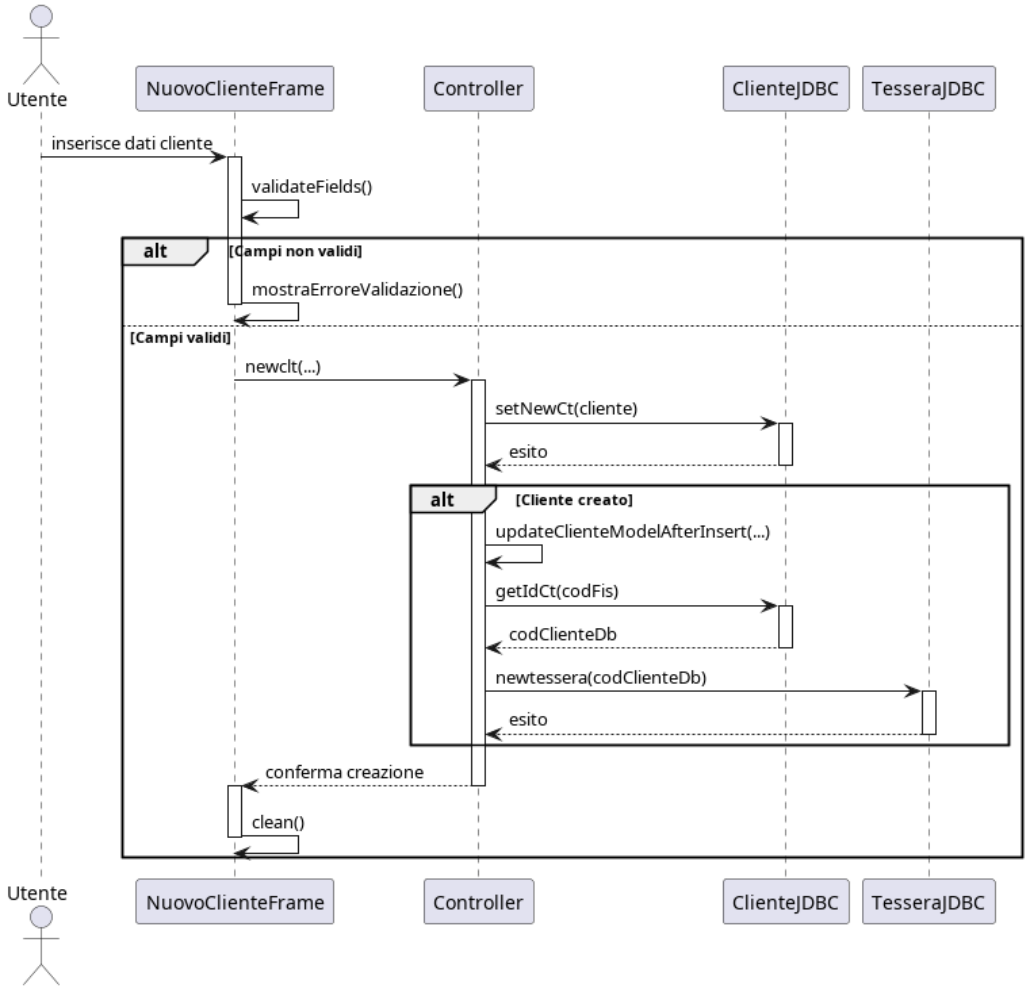

Figura 3: Diagramma di sequenza: Nuovo Cliente